

Software Technology 2 | Assignment 2

Data Structures and Algorithms (DSA) Explorer and Visualiser

1. Project Overview

This project is an interactive Data Structures and Algorithms (DSA) Explorer and Visualiser built using Python 3 and the Pygame library. The app lets users explore common data structures and algorithms step by step through a visual game-like interface.

The app is split into three phases:

- Phase 1 - Data Structures Playground: Stack, Queue, Linked List, Binary Search Tree
- Phase 2 - Algorithm Visualiser: Bubble/Selection/Merge Sort, BFS/DFS Graph Traversal, Max-Heap
- Phase 3 - Puzzle Challenges: Dijkstra pathfinding, A* pathfinding, Event Queue Simulator, Dynamic Programming Grid

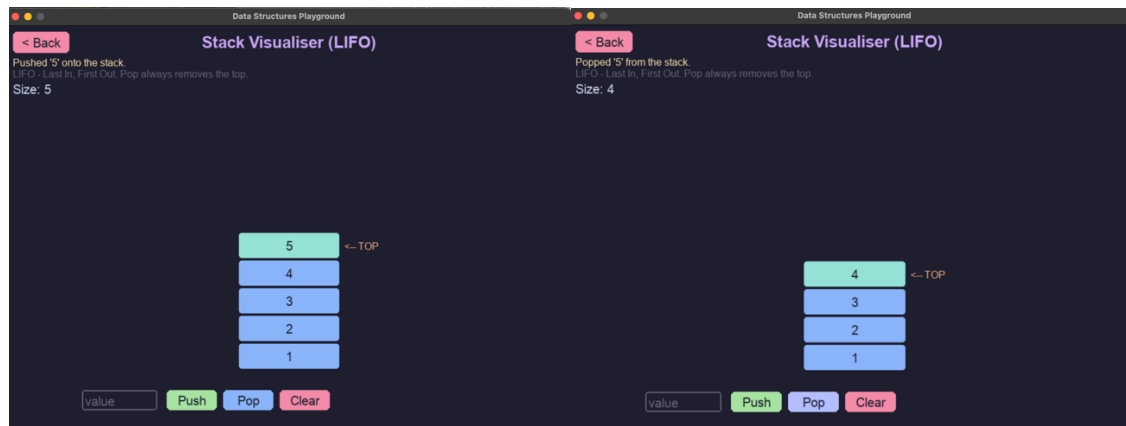
All modules are accessible from a main menu. Each has its own Pygame screen and a Back button to return to the menu.

2. Phase 1 - Data Structures Playground

2.1 Stack

The stack is drawn as a column of blocks showing LIFO (Last In, First Out) order. The top item is highlighted so it is obvious which one will be popped next.

- Push - type a value and click Push
- Pop - removes the top item, shows a message if the stack is empty
- Peek - shows the top item without removing it
- Clear - empties the stack



2.2 Queue

The queue is drawn as a horizontal row of boxes. The front is coloured differently to the rear so the direction of flow is clear. FIFO (First In, First Out) order is enforced.

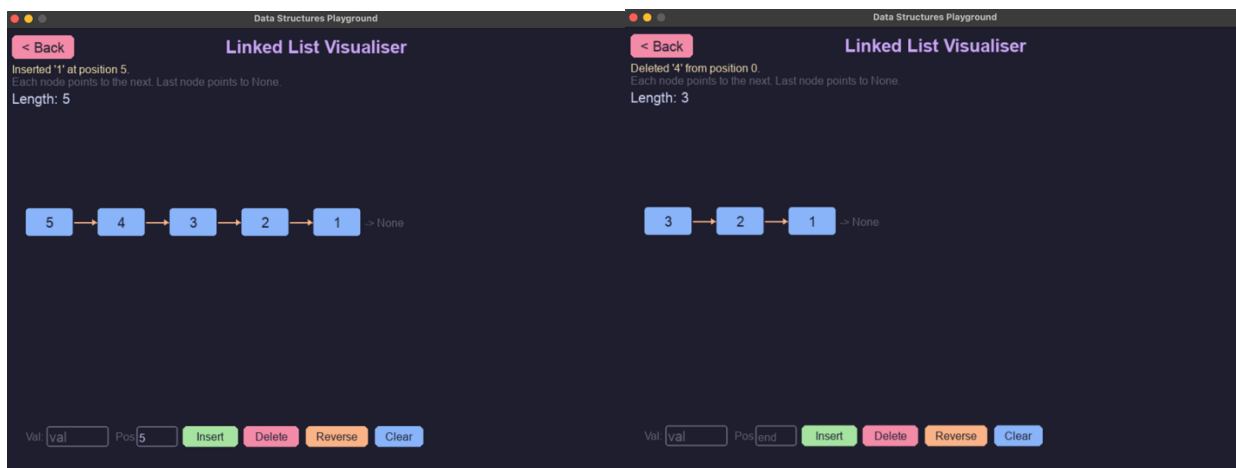
- Enqueue - adds to the rear
- Dequeue - removes from the front



2.3 Linked List

Nodes are drawn as boxes connected by arrows. Supports inserting at a specific position, deleting by position, and reversing the whole list.

- Insert - adds a node at a given position
- Delete - removes a node at a given position and returns its value
- Reverse - flips the list using the standard three-pointer approach

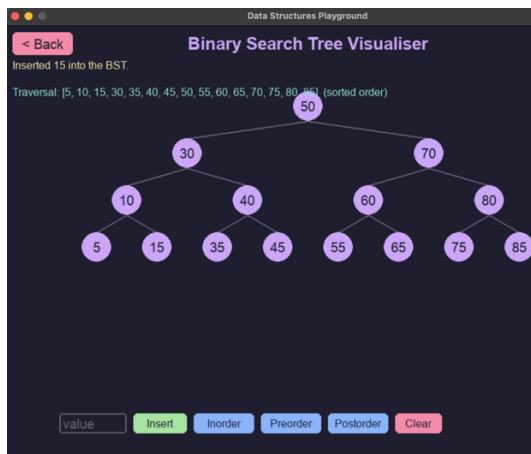


2.4 Binary Search Tree (BST)

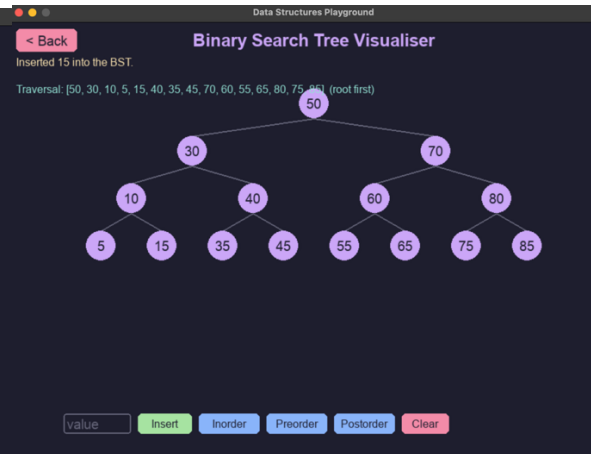
Nodes are drawn as circles with connecting lines. The user inserts integers and then steps through the three traversal orders one node at a time.

- Insert - integers only, duplicates are ignored
- In-order (Left, Root, Right) - always produces sorted output
- Pre-order (Root, Left, Right)
- Post-order (Left, Right, Root)

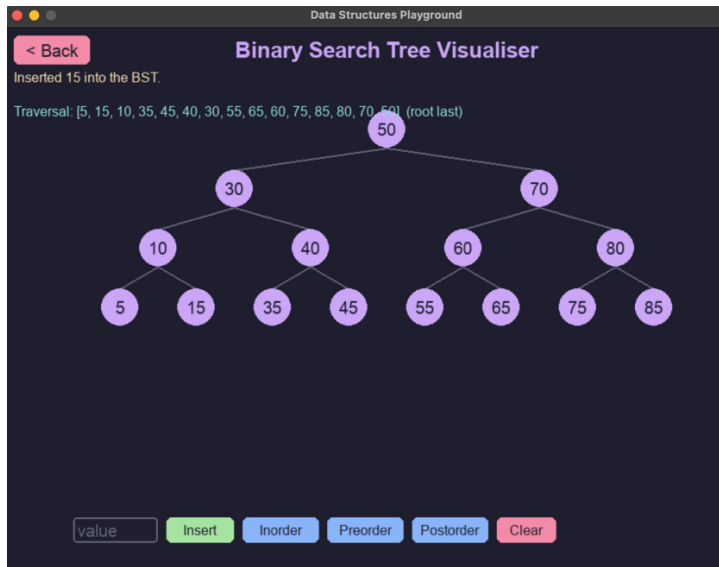
In-Order:



Pre-Order:



Post-Order:

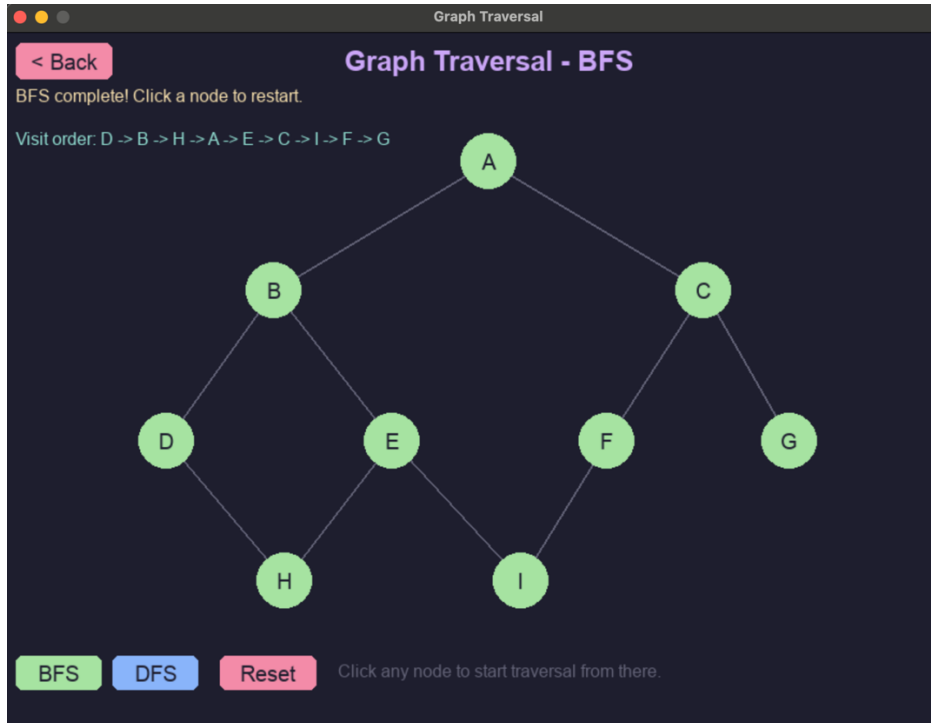


3. Phase 2 - Algorithm Visualiser

3.1 Graph Traversal (BFS and DFS)

The user builds a graph by adding edges, picks a start node, then steps through BFS or DFS. Visited nodes change colour as the traversal progresses.

BFS uses a queue (deque) - visits nodes level by level. DFS uses a stack - goes as deep as possible before backtracking.



3.2 Sorting Algorithms

All three algorithms display as a bar chart where height = value. Steps are generated upfront so the user can step forward/back or use auto-play.

Bubble Sort $O(n^2)$ - compares adjacent pairs, swapping if out of order. Comparing bars turn one colour, swapping bars turn another.



Selection Sort $O(n^2)$ - finds the minimum on each pass and swaps it into place. The current minimum is highlighted.



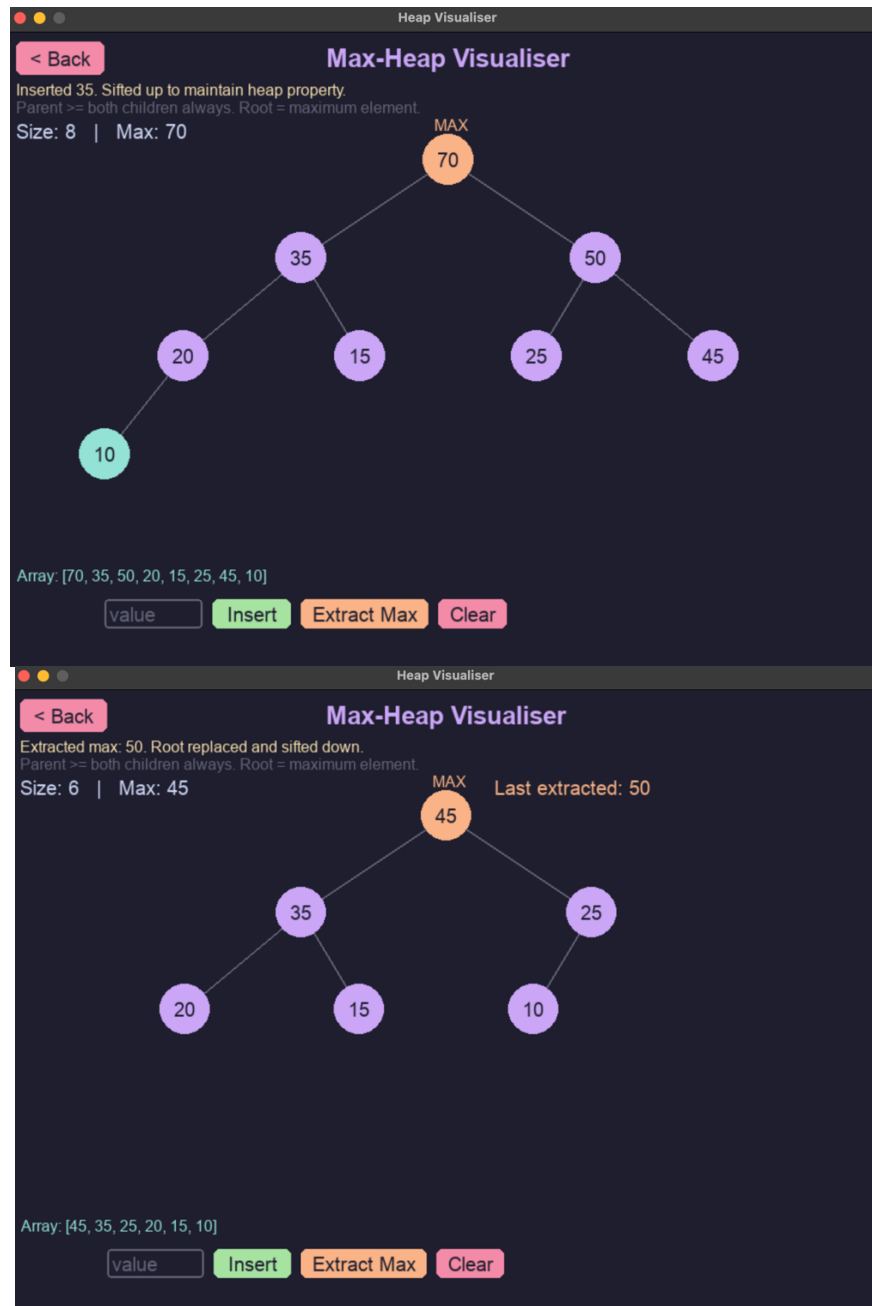
Merge Sort $O(n \log n)$ - bottom-up implementation. The range being merged is highlighted.



3.3 Max-Heap Visualiser

The heap is shown as a tree. The root is always the maximum. Insert triggers a sift-up animation and extract-max triggers a sift-down animation.

- Insert - adds a value and sifts it up to restore the heap property
- Extract Max - removes the root and sifts the replacement value down

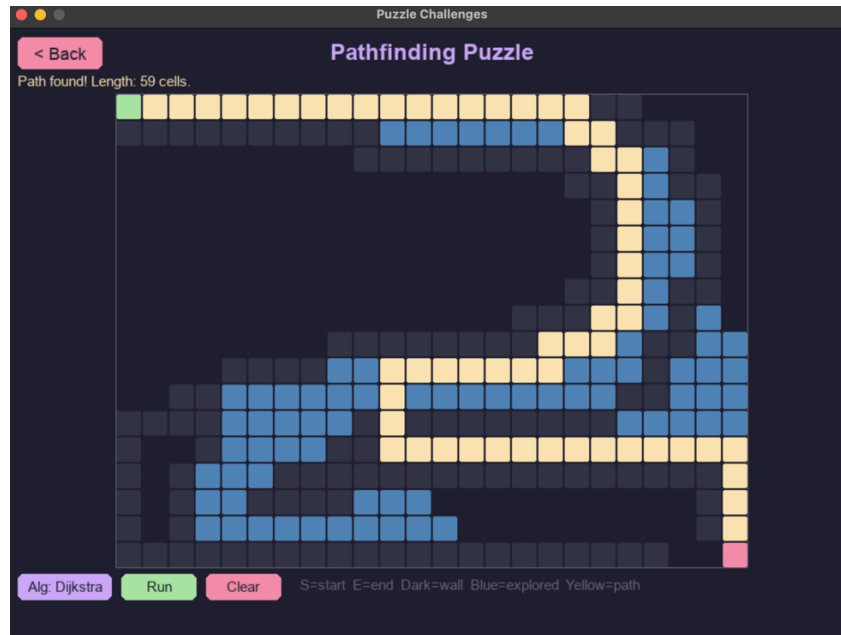


4. Phase 3 - Puzzle Challenges

4.1 Dijkstra Pathfinding

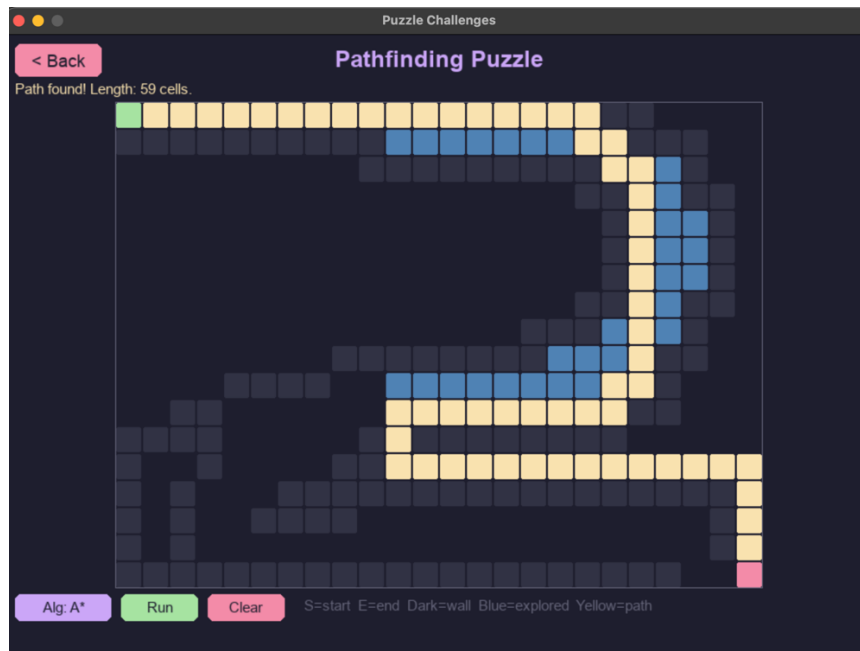
A grid is shown where the user can paint walls. Dijkstra's algorithm then finds the shortest path from start to end, visualising the visited nodes as it expands.

Uses a min-heap priority queue. Explores nodes in order of their distance from the start. Guaranteed to find the shortest path.



4.2 A* Pathfinding

Same grid setup as Dijkstra but uses a heuristic (Manhattan distance) to guide the search towards the goal. Generally faster than Dijkstra because it avoids exploring irrelevant nodes.



4.3 Event Queue Simulator

This puzzle demonstrates a priority queue backed by a Max-Heap. The user adds events with a name and a priority number (0-99). Events are processed in priority order - the highest priority event is always processed first, like how a hospital triage system works.

This directly uses the MaxHeap from Phase 2 (heap_logic.py). The extract_max operation ensures the highest priority event is always processed next, which is the core heap property applied to a real-world scheduling scenario.

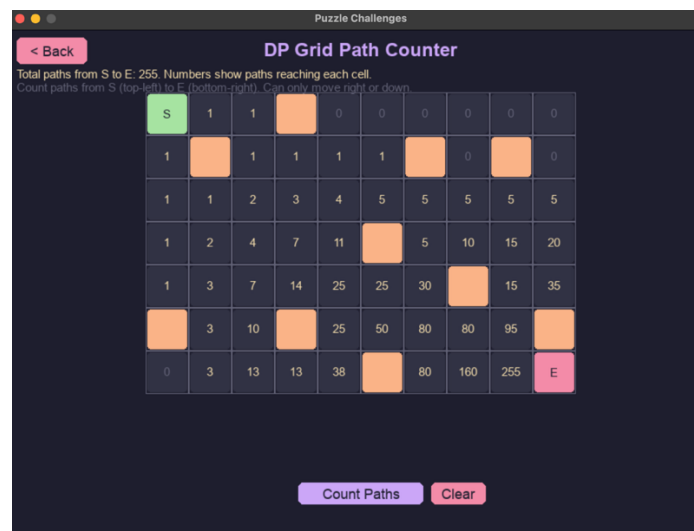


4.4 Dynamic Programming Grid

The user toggles obstacle cells on a grid, then clicks Solve. The DP table fills in cell by cell counting the number of unique paths from top-left to bottom-right.

Recurrence relation:

$$\begin{aligned} dp[i][j] &= dp[i-1][j] + dp[i][j-1] && \text{(if not blocked)} \\ dp[i][j] &= 0 && \text{(if blocked)} \end{aligned}$$



5. Testing

5.1 test_phase1.py - Data Structures (26 tests)

Covers Stack, Queue, Linked List, and BST.

- Stack: push increases size, pop returns last pushed (LIFO), pop empty returns None, peek does not remove
- Queue: enqueue/dequeue maintains FIFO order, dequeue empty returns None, to_list correct order
- Linked List: insert at position, delete returns correct value, reverse flips order
- BST: inorder gives sorted output, preorder visits root first, postorder visits root last, search works, duplicates ignored

```
test_bst_insert_and_inorder_traversal (__main__.TestBST.test_bst_insert_and_inorder_traversal)
Insert [50, 30, 70] and verify inorder traversal is 30, 50, 70. ... ok
test_duplicates_ignored (__main__.TestBST.test_duplicates_ignored)
Inserting the same value twice should not add duplicates. ... ok
test_inorder_gives_sorted (__main__.TestBST.test_inorder_gives_sorted)
Inorder traversal of a BST always gives sorted order. ... ok
test_postorder (__main__.TestBST.test_postorder)
Postorder visits root last. ... ok
test_preorder (__main__.TestBST.test_preorder)
Preorder visits root first. ... ok
test_search_found (__main__.TestBST.test_search_found)
Search should find values that were inserted. ... ok
test_search_not_found (__main__.TestBST.test_search_not_found)
Search should return False for values not in the tree. ... ok
test_delete_empty_returns_none (__main__.TestLinkedList.test_delete_empty_returns_none)
Deleting from empty list should return None. ... ok
test_delete_node (__main__.TestLinkedList.test_delete_node)
Deleting at position 1 should remove the correct node. ... ok
test_insert_at_end (__main__.TestLinkedList.test_insert_at_end)
Inserting without a position should append to the end. ... ok
test_insert_at_position (__main__.TestLinkedList.test_insert_at_position)
Insert node with value 10 at position 2. ... ok
test_reverse (__main__.TestLinkedList.test_reverse)
Reversing the list should flip the order. ... ok
test_dequeue_empty_returns_none (__main__.TestQueue.test_dequeue_empty_returns_none)
Dequeuing from empty queue should return None. ... ok
test_enqueue_dequeue_fifo (__main__.TestQueue.test_enqueue_dequeue_fifo)
Queue is FIFO - first in, first out. ... ok
test_enqueue_dequeue_four_three (__main__.TestQueue.test_enqueue_dequeue_four_three)
Enqueue 4 items, dequeue 3 - FIFO order maintained. ... ok
test_size_tracks_correctly (__main__.TestQueue.test_size_tracks_correctly)
Size should update with enqueue and dequeue. ... ok
test_to_list_order (__main__.TestQueue.test_to_list_order)
to_list should reflect the queue order front-to-back. ... ok
test_is_empty (__main__.TestStack.test_is_empty)
New stack should be empty. ... ok
test_peek_does_not_remove (__main__.TestStack.test_peek_does_not_remove)
Peek should show the top item without removing it. ... ok
test_pop_empty_returns_none (__main__.TestStack.test_pop_empty_returns_none)
Popping from an empty stack should return None, not crash. ... ok
test_pop_returns_last_pushed (__main__.TestStack.test_pop_returns_last_pushed)
LIFO - the last thing pushed should be the first thing popped. ... ok
test_push_increases_size (__main__.TestStack.test_push_increases_size)
Pushing items should increase the stack size. ... ok
test_push_pop_sequence (__main__.TestStack.test_push_pop_sequence)
Push 3 items, pop 2 - stack should have 1 item left in correct order. ... ok
```

5.2 test_phase2.py - Sorting, Graph, Heap (21 tests)

Covers all three sorting algorithms, graph traversal, and the MaxHeap.

- Bubble sort: sorts correctly, handles already sorted and reverse sorted input, steps yield valid actions
- Selection sort: sorts correctly, compare indices are always valid positions
- Merge sort: sorts correctly, empty array handled, original array not mutated
- Graph: BFS and DFS visit all nodes, BFS visits start first, no duplicate edges
- Heap: peek returns max, extract gives descending order, heap property maintained after every insert

```
test_already_sorted (__main__.TestBubbleSort.test_already_sorted)
Already sorted arrays should stay the same. ... ok
test_reverse_sorted (__main__.TestBubbleSort.test_reverse_sorted)
Worst case - reverse sorted input. ... ok
test_single_element (__main__.TestBubbleSort.test_single_element)
Single-element array should be unchanged. ... ok
test_sorts_correctly (__main__.TestBubbleSort.test_sorts_correctly)
Bubble sort should sort [5,3,8,1,2] to [1,2,3,5,8]. ... ok
test_steps_yield_compare_or_swap (__main__.TestBubbleSort.test_steps_yield_compare_or_swap)
Every step should have action 'compare', 'swap', or 'done'. ... ok
test_add_and_check_edge (__main__.TestGraph.test_add_and_check_edge)
Adding an edge should make both nodes neighbours of each other. ... ok
test_bfs_visits_all_nodes (__main__.TestGraph.test_bfs_visits_all_nodes)
BFS from A should eventually visit all reachable nodes. ... ok
test_bfs_visits_in_correct_order (__main__.TestGraph.test_bfs_visits_in_correct_order)
BFS from A should visit A first (it's the start). ... ok
test_dfs_order_matches_expected (__main__.TestGraph.test_dfs_order_matches_expected)
DFS from A - A should be the first visited. ... ok
test_dfs_visits_all_nodes (__main__.TestGraph.test_dfs_visits_all_nodes)
DFS from A should also visit all nodes. ... ok
test_no_duplicate_edges (__main__.TestGraph.test_no_duplicate_edges)
Adding the same edge twice should not create duplicates. ... ok
test_extract_from_empty (__main__.TestHeap.test_extract_from_empty)
Extracting from an empty heap should return None. ... ok
test_extract_max_order (__main__.TestHeap.test_extract_max_order)
Extracting max repeatedly should give values in descending order. ... ok
test_heap_property_maintained (__main__.TestHeap.test_heap_property_maintained)
After every insert, the parent should be >= its children. ... ok
test_insert_and_peek_max (__main__.TestHeap.test_insert_and_peek_max)
After inserting values the root (peek) should always be the max. ... ok
test_size_decreases_after_extract (__main__.TestHeap.test_size_decreases_after_extract)
Size should go down by one each time we extract. ... ok
test_empty_array (__main__.TestMergeSort.test_empty_array)
Empty array should return empty. ... ok
test_original_not_mutated (__main__.TestMergeSort.test_original_not_mutated)
The original array should not be changed by the generator. ... ok
test_sorts_correctly (__main__.TestMergeSort.test_sorts_correctly)
Merge sort should correctly sort any array. ... ok
test_highlights_during_compare (__main__.TestSelectionSort.test_highlights_during_compare)
During a compare step, the two indices being compared ... ok
test_sorts_correctly (__main__.TestSelectionSort.test_sorts_correctly)
Selection sort should produce the same sorted result. ... ok
```

5.3 test_phase3.py - Pathfinding and DP (14 tests)

Covers Dijkstra, A*, and the DP grid path counter.

- Dijkstra: finds path on open grid, returns None when goal is blocked, path cells are always adjacent, path avoids walls
- A*: finds same length path as Dijkstra on open grid, returns None when blocked, correct path length on small grid
- DP grid: 3x3 open grid has 6 paths, 1x1 grid has 1 path, walls reduce path count, table dimensions match grid

```
test_finds_same_path_as_dijkstra (__main__.TestAStar.test_finds_same_path_as_dijkstra)
On an unweighted grid, A* and Dijkstra should find the same path length. ... ok
test_no_path_when_blocked (__main__.TestAStar.test_no_path_when_blocked)
A* should also return no path when goal is unreachable. ... ok
test_path_is_correct_length_on_small_grid (__main__.TestAStar.test_path_is_correct_length_on_small_grid)
On a 3x3 open grid, shortest path from (0,0) to (2,2) needs 5 cells. ... ok
test_all_walls_except_edges_still_finds_path (__main__.TestDPGridPaths.test_all_walls_except_edges_still_finds_path)
A path along just the edges should still be counted. ... ok
test_dp_table_shape (__main__.TestDPGridPaths.test_dp_table_shape)
The returned DP table should have the same dimensions as the grid. ... ok
test_no_path_when_fully_blocked (__main__.TestDPGridPaths.test_no_path_when_fully_blocked)
If start or first row/col is fully walled off, count should be 0. ... ok
test_open_grid_path_count (__main__.TestDPGridPaths.test_open_grid_path_count)
On a 3x3 open grid, there are 6 paths from top-left to bottom-right. ... ok
test_single_cell_grid (__main__.TestDPGridPaths.test_single_cell_grid)
1x1 grid has exactly 1 path (you're already at the end). ... ok
test_wall_blocks_paths (__main__.TestDPGridPaths.test_wall_blocks_paths)
Placing a wall should reduce the number of paths. ... ok
test_finds_path_on_open_grid (__main__.TestDijkstra.test_finds_path_on_open_grid)
Dijkstra should find a path on a completely open grid. ... ok
test_no_path_when_blocked (__main__.TestDijkstra.test_no_path_when_blocked)
When the goal is completely surrounded by walls, no path should exist. ... ok
test_path_avoids_walls (__main__.TestDijkstra.test_path_avoids_walls)
The path should never go through a wall cell. ... ok
test_path_cells_are_adjacent (__main__.TestDijkstra.test_path_cells_are_adjacent)
Every step in the path should be exactly one move away from the previous. ... ok
test_path_starts_and_ends_correctly (__main__.TestDijkstra.test_path_starts_and_ends_correctly)
Path should start at start and end at end. ... ok
```

6. Benchmarking

6.1 Phase 1 - Data Structure Benchmarks

Stack, Queue, Linked List, and BST are all timed for $n = 100$, 1000 , and 10000 operations. All complete in under a second at these sizes since the operations are $O(1)$ or $O(n)$.

```
test_bst_benchmark (__main__.BenchmarkPhase1.test_bst_benchmark) ...
--- BST Benchmark ---
n=100    insert all + inorder -> 0.000136s
n=500    insert all + inorder -> 0.000675s
n=1000   insert all + inorder -> 0.001417s
ok
test_linked_list_benchmark (__main__.BenchmarkPhase1.test_linked_list_benchmark) ...
--- Linked List Benchmark ---
n=100    insert all + reverse -> 0.000233s
n=500    insert all + reverse -> 0.005016s
n=1000   insert all + reverse -> 0.018427s
ok
test_queue_benchmark (__main__.BenchmarkPhase1.test_queue_benchmark) ...
--- Queue Benchmark ---
n=100    enqueue+dequeue all -> 0.000025s
n=1000   enqueue+dequeue all -> 0.000179s
n=10000  enqueue+dequeue all -> 0.001856s
ok
test_stack_benchmark (__main__.BenchmarkPhase1.test_stack_benchmark) ...
--- Stack Benchmark ---
n=100    push+pop all -> 0.000026s
n=1000   push+pop all -> 0.000178s
n=10000  push+pop all -> 0.001754s
ok
```

6.2 Phase 2 - Sorting Benchmarks

BFS and DFS are timed on graphs of 10, 50, and 100 nodes. Both run in under a millisecond at these sizes.

MaxHeap insert and extract are timed at $n = 100$, 1000 , and 5000 . Extract runs in $O(\log n)$ time so it stays fast even at 5000 elements.

Bubble and Selection sort are tested at $n = 50$, 100 , 300 . Merge sort is tested at $n = 100$, 500 , 1000 . A direct comparison test confirms that merge sort is faster than bubble sort at $n = 200$, which matches the expected $O(n \log n)$ vs $O(n^2)$ complexity difference.

```
test_bfs_benchmark (__main__.BenchmarkPhase2Graph.test_bfs_benchmark) ...
--- BFS Benchmark ---
  nodes=10    -> 0.000022s
  nodes=50    -> 0.000072s
  nodes=100   -> 0.000163s
ok
test_bfs_vs_dfs_same_result (__main__.BenchmarkPhase2Graph.test_bfs_vs_dfs_same_result)
BFS and DFS should visit the same set of nodes, just in different order. ... ok
test_dfs_benchmark (__main__.BenchmarkPhase2Graph.test_dfs_benchmark) ...
--- DFS Benchmark ---
  nodes=10    -> 0.000014s
  nodes=50    -> 0.000074s
  nodes=100   -> 0.000180s
ok
test_heap_extract_benchmark (__main__.BenchmarkPhase2Heap.test_heap_extract_benchmark) ...
--- MaxHeap Extract Benchmark ---
  n=100    extract all -> 0.000129s
  n=1000   extract all -> 0.002204s
  n=5000   extract all -> 0.012768s
ok
test_heap_insert_benchmark (__main__.BenchmarkPhase2Heap.test_heap_insert_benchmark) ...
--- MaxHeap Insert Benchmark ---
  n=100    insert all -> 0.000040s
  n=1000   insert all -> 0.000383s
  n=5000   insert all -> 0.001949s
ok
test_bubble_sort_benchmark (__main__.BenchmarkPhase2Sorting.test_bubble_sort_benchmark) ...
--- Bubble Sort Benchmark ---
  n=50     -> 0.000517s
  n=100    -> 0.003407s
  n=300    -> 0.052271s
ok
test_merge_sort_benchmark (__main__.BenchmarkPhase2Sorting.test_merge_sort_benchmark) ...
--- Merge Sort Benchmark ---
  n=100    -> 0.000764s
  n=500    -> 0.032911s
  n=1000   -> 0.198665s
ok
test_selection_sort_benchmark (__main__.BenchmarkPhase2Sorting.test_selection_sort_benchmark) ...
--- Selection Sort Benchmark ---
  n=50     -> 0.000311s
  n=100    -> 0.001900s
  n=300    -> 0.032840s
ok
test_sorting_comparison (__main__.BenchmarkPhase2Sorting.test_sorting_comparison) ...
--- Sorting Comparison n=200 ---
  Bubble Sort -> 0.016507s
  Merge Sort  -> 0.002597s
ok
```

6.3 Phase 3 - Pathfinding Benchmarks

Dijkstra and A* are timed on grids of 5x5, 10x10, and 20x20. A comparison test on a 15x15 grid confirms that A* is not significantly slower than Dijkstra and is often faster because the heuristic reduces the number of nodes explored.

Benchmarking for the Event Queue Simulator was not included in test_benchmarks.py as the simulator is an interactive Pygame-based puzzle dependent on user input, making automated timing tests impractical. The underlying MaxHeap performance is covered by the heap benchmarks in 6.2 Phase 2.

The DP grid counter is timed at 5x5, 10x10, 20x20, and 50x50. It runs in $O(\text{rows} \times \text{cols})$ time so even the 50x50 case finishes almost instantly.

```
test_astar_benchmark (__main__.BenchmarkPhase3.test_astar_benchmark) ...
--- A* Benchmark ---
  grid=5x5   -> 0.000109s
  grid=10x10  -> 0.000268s
  grid=20x20  -> 0.001636s
ok
test_astar_faster_than_dijkstra (__main__.BenchmarkPhase3.test_astar_faster_than_dijkstra) ...
--- Pathfinding Comparison 15x15 grid ---
  Dijkstra   -> 0.000644s
  A*         -> 0.000683s
ok
test_dijkstra_benchmark (__main__.BenchmarkPhase3.test_dijkstra_benchmark) ...
--- Dijkstra Benchmark ---
  grid=5x5   -> 0.000052s
  grid=10x10  -> 0.000227s
  grid=20x20  -> 0.001511s
ok
test_dp_grid_benchmark (__main__.BenchmarkPhase3.test_dp_grid_benchmark) ...
--- DP Grid Path Count Benchmark ---
  grid=5x5   -> 0.000009s
  grid=10x10  -> 0.000014s
  grid=20x20  -> 0.000047s
  grid=50x50  -> 0.000289s
ok
```